Operating Systems — Homework 3

Ex. 3.1 (10 pts) — loading the OS and one program without an MMU

The constraint. With no MMU there is no translation step: the address the CPU puts on the bus is the physical address. The program is linked to begin at 0 and to run over a contiguous range, so physical address 0 and the bytes above it must belong to the application, and the OS cannot have them.

The layout. Load the OS at the high end of RAM, growing downward from the physical top, and the application at physical 0, growing upward: text, then data and bss, then the heap, with the stack at the top of the user region growing down (Figure 1). The program owns the whole of $[0, F)$, where F is the bottom of the OS, so both of the CPU's assumptions hold with no translation and no relocation: the loader copies the executable to address 0 byte for byte. Running a different program overwrites $[0, F)$; the OS never moves.

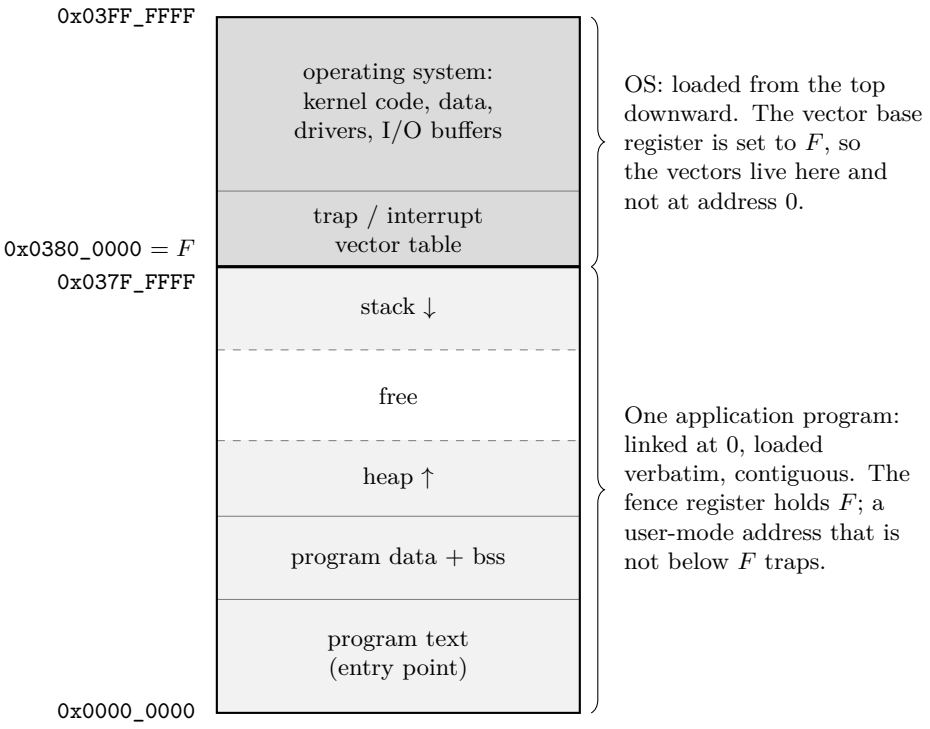


Figure 1: Memory map for a single-program system with no MMU. Physical memory is 64 MiB and the OS occupies the top 8 MiB. Addresses increase upward.

Address 0 is not usually free. Most CPUs fetch trap and interrupt vectors from fixed low addresses: the 68000's table at `0x000`–`0x3FF`, x86 real mode's IVT at `0x00000`–`0x003FF`, ARM's vectors at 0. That collides with the layout, so I require a CPU with a vector base register (68010 VBR, ARM VBAR, RISC-V `mtvec`) and set it to F. Such a register is consulted only when an

exception is dispatched and leaves ordinary loads and stores untouched, so it is not an MMU. On a CPU that hard-wires its vectors at 0 the assumption cannot be met: the vectors keep the low addresses and the program has to be relocated when it is loaded. Only the reset vector cannot be moved in principle, and it is served from boot ROM.

The boundary. No MMU also means no permission check per access, so nothing above stops the application from storing into the OS. I therefore allow one fence (limit) register holding F: in user mode the CPU traps any address that is not below F. A comparison adds nothing to the address, so this is still not translation; a *base* register, added to every address, is where an MMU begins.

Ex. 3.2 (10 pts) — one giant fast memory and the hierarchy

No. The hierarchy would survive, because the premise of the question fails: a device's access time is set by its material *and* its size, and at terabyte scale it is size that dominates.

Every read must physically cross the array: the address propagates through the decoder to a word line, and the data returns along a bit line through a sense amplifier. That time is bounded below by the time needed to cross the array, whose linear dimension grows as $\sqrt{C}$ in the capacity C. Take the densest cell in production, a 6T SRAM bit at $0.021\,\mu\text{m}^2$; the faster high-performance variant is larger at $0.025\,\mu\text{m}^2$, so this keeps the bound conservative. One terabyte is 8×10^{12} cells covering $1.68 \times 10^5\,\text{mm}^2$, about 2.4 whole 300 mm wafers and some 0.41 m on a side. Crossing that and returning takes at least 2.7 ns *in vacuum*, eleven clock cycles at 4 GHz, before a single transistor switches; real on-chip wires are several times slower than c. A terabyte of the fastest material is thus nowhere near as fast as a kilobyte of it.

The penalty is already visible within one technology. L1, L2 and L3 on a modern CPU are all SRAM, on the same die at the same process node, yet they answer in roughly 4, 14 and 40 cycles. Only capacity differs: longer word and bit lines carry more RC delay, deeper decoders add stages, and larger bit-line capacitance makes sense amplifiers resolve more slowly. So the hierarchy is not an artifact of mixing cheap slow memory with expensive fast memory. It reappears with the material held fixed.

Power says the same thing. The fastest cells are the leakiest, six transistors per bit with static current in each and no idle state as cheap as a DRAM capacitor's. A terabyte of them could not be powered or cooled as one block, and spreading it out to cool it pays the propagation delay back again.

The right use of such a device would therefore be to put a small fast array in front of it to absorb the references that locality makes predictable: to build the hierarchy inside it. Cost per bit would stop being the motive; the device's own size–speed trade-off would supply one.

Ex. 3.3 (5 pts) — servicing a fault when every frame is in use

No frame is free, so the page cannot be read in until the OS takes one away. The fault handler must:

1. **Choose a victim frame**, using a replacement policy (Ex. 3.8).
2. **Clean the victim if it is dirty.** Test its modified bit M. If $M = 1$ the frame no longer matches its copy on disk and must be written to the swap area first; if $M = 0$ that copy is still valid and the frame can be taken at once.
3. **Break the old mapping.** Mark the victim's page-table entry not-present in its owner's page table *and* invalidate the matching TLB entry, or a stale translation will keep reaching a frame that now holds a different page.

4. **Read the wanted page** from disk into the freed frame. The faulting process blocks for the duration and the CPU goes to another process.
5. **Install the new mapping**: frame number into the faulting entry, present bit set, R and M cleared, protection bits set.
6. **Back up and restart the faulting instruction** (Ex. 3.7).

Steps 2 and 4 are each a disk access, so a fault that must clean its victim costs about twice one that need not; systems therefore keep a pool of pre-cleaned free frames (Ex. 3.6) to move the write-back off the fault path.

Ex. 3.4 (5 pts) — why NFU over LRU

Cost, and often availability. Exact LRU has to know the order of *every* reference, and a reference that hits a valid page-table entry is resolved by the MMU without the OS ever seeing it. True LRU must therefore be built in hardware: a counter or clock value written into the entry on every access, with a scan of all frames at eviction time to find the minimum; or a list moved-to-front on every access; or an $n \times n$ matrix of bits updated per reference. Such specialised hardware is rare, so on most machines exact LRU is unavailable at any price, and where it exists it taxes every memory access.

NFU, as defined in Ex. 3.8, needs only the reference bit that essentially every MMU already maintains for free. Its work happens once per clock tick — add R to the page's counter, clear R — which is $O(\text{frames})$ per tick instead of $O(1)$ per reference, and it is ordinary software that ports to any MMU with an R bit.

What is given up: NFU ranks pages by how *often* they were used over their whole lifetime rather than how *recently*, and cannot order two references in the same tick. It is the right choice when LRU's hardware is missing or its per-reference cost unaffordable.

Ex. 3.5 (5 pts) — why NFU with aging over NFU or LRU

Over plain NFU. NFU's counter is a lifetime total that is never reduced, so the algorithm never forgets. A page referenced heavily during start-up keeps a large count permanently and is protected long after it has gone cold, while a page that has only just become busy has a small count and is evicted in its place. Pages that were all once busy accumulate large, indistinguishable counts with no basis for choosing between them.

Aging keeps the per-tick sampling but changes the arithmetic: each tick the counter is shifted right one bit and R is inserted at the *high-order* end. A reference's contribution therefore halves every tick, so the counter is dominated by recent history and the position of its highest set bit says how long ago the page was last touched. The smallest counter is now approximately the least recently used page: recency rather than lifetime frequency.

Over LRU. The cost argument of Ex. 3.4 is unchanged: a reference bit, one shift and one OR per page per tick, all in software.

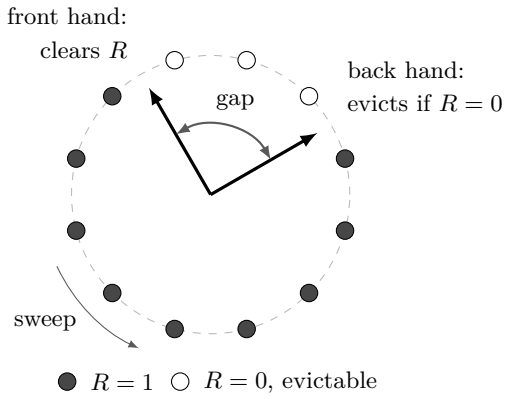
What it still cannot do. The counter has finite width, typically 8 bits, so pages untouched for more than 8 ticks all read zero and cannot be separated, and two references within one tick look like one. Aging approximates LRU; it does not implement it.

Ex. 3.6 (5 pts) — what the two-handed clock is for

Its purpose is to separate *clearing* reference bits from *evicting* pages, so that the rate of each can be set independently.

With one hand the same pointer does both jobs, so how fast it turns is set entirely by the eviction rate. On a machine with a lot of memory a revolution takes so long that by the time the hand returns nearly every page has been touched again; it finds $R = 1$ almost everywhere, clears bits and sweeps on, and the page it finally takes is chosen by position in the frame list rather than by recent use — FIFO behaviour. Driving it faster fails the other way, leaving pages too little time to set R so that nearly all of them look unreferenced.

Two hands fix the window. The front hand clears R; the back hand, a fixed distance behind, releases any page whose R is still clear, so it went unreferenced while the gap was traversed. The gap is the test interval, a tunable parameter rather than a side effect of memory size and load.



BSD's pageout daemon uses this to keep a free-frame pool: woken periodically, it sweeps only as far as needed to restore `lotsfree` free frames, so a fault normally finds a clean frame waiting instead of paying for a scan and a write-back (Ex. 3.3).

Ex. 3.7 (10 pts) — backing up the instruction after a page fault

Which instruction. The one that was executing when the fault was taken — the faulting instruction itself. It must be re-executed from its first byte, so the PC saved by the trap has to be backed up to the start of that instruction. Not the one before it, and not resumed from the middle.

Why it has to be backed up. *The memory access never happened.* The instruction was abandoned at the operand reference that missed, so its effect was never produced; returning to the following instruction would silently skip a load or a store. Paging must be invisible to the program, and the only way to make the access occur is to execute the instruction again.

The saved PC need not point at the start of it. On a variable-length CISC encoding the PC advances as the instruction is decoded, so when the fault is taken it may already be past the opcode and some operand bytes; on the 68000 it points just after the part that faulted. The handler cannot recover the start by scanning backwards, since with variable-length encodings there is no telling where the preceding instruction began, so the hardware must supply the address.

Side effects may already be committed. Auto-increment and auto-decrement modes update their index register as a side effect, and a block move copies as it goes. A fault partway through leaves registers already stepped and part of the block written. Re-executing from the start applies those updates twice; resuming skips the transfer. Either way the program gets a wrong answer unless the partial effects are undone first.

So machines either record enough internal state for microcode or the handler to unwind what was done and recompute the starting address, or define their instructions to be restartable:

every fault detected before any register is altered, with the faulting address kept in a dedicated register, as RISC machines do.

Ex. 3.8 (10 pts, 2 pts each) — five page replacement algorithms

FIFO. Keep the resident pages on a queue in load order; evict the one at the head, which has been resident longest, and append the arrival at the tail. One pointer, no hardware support, and no usage information at all — so a page in constant use is discarded simply for being old. FIFO is not a stack algorithm and can show Belady's anomaly, where giving it more frames *increases* the number of faults.

FILO (first-in last-out). The complement, as the name is used here: the resident pages form a stack and the victim is the most recently loaded one, so arrivals keep reusing the same frame. As cheap as FIFO and far worse, because the newest page is precisely the one the current locality has just demanded; it is thrown out at once while the rest of memory never turns over.

NRU (not recently used). Sort pages by their (R, M) bits into four classes, numbered 0 to 3 for $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$, and evict an arbitrary page from the lowest-numbered non-empty class. R is cleared on every clock tick, so it means "referenced since the last tick", while M is left alone. Cheap, and it prefers a clean unreferenced page, which needs no write-back.

LRU. Evict the page that has gone unreferenced longest, on the assumption that recent use predicts imminent use. Exact implementation needs work on every reference: a timestamp written into the page-table entry at each access, a list reordered at each access, or an $n \times n$ matrix of bits. LRU is a stack algorithm and so is immune to Belady's anomaly; its cost is why the approximations exist.

NFU (not frequently used). One software counter per page. At each clock tick the OS adds that page's R bit to its counter and clears R, and the page with the smallest counter is evicted. It needs only the reference bit, but it measures frequency over a page's entire lifetime and never forgets, so a page that was busy early keeps a high score after going cold — the flaw that aging removes (Ex. 3.5).

Ex. 3.9 (20 pts) — three papers from 'Memory Management'

I read papers 3-1, 3-6 and 3-4, chosen because between them they follow one idea through three eras: the behavioural model that told an OS what to keep resident, the kernel structure that made paging portable, and the hardware pressure now reshaping both. (The list truncates 3-6's title and mistypes 3-2's; the citations below are the published ones.)

1. The working set model (paper 3-1)

Peter J. Denning, "The Working Set Model for Program Behavior", *Communications of the ACM* **11**(5):323–333, May 1968. First presented at the ACM Symposium on Operating System Principles, Gatlinburg, TN, October 1967.

Summary. Resource allocation had no theory, Denning argues, because program behaviour had no model, and the model cannot come from outside the program: users cannot cost shared, data-dependent code, and compilers cannot see across separately compiled modules. The system must measure behaviour itself.

He defines the working set $W(t, \tau)$ as the pages a process referenced during the process-time interval $(t - \tau, t)$, with size $\omega(t, \tau)$. It is monotone increasing and concave in τ, with $\omega(t, 2\tau) \leq 2\omega(t, \tau)$ on average; $W(t, \tau)$ predicts $W(t + \alpha, \tau)$ for α small against τ but not for $\alpha \gg \tau$; and from the distribution F_x of the interval between successive references to a page, pages re-enter the working set at rate $\lambda(\tau) = [1 - F_x(\tau)]/\bar{x}$, an estimate of page traffic. Setting τ

trades that traffic against wasted memory, and since a page evicted just before re-reference costs two traverse times to return, he recommends a τ comparable to the traverse time T, $\tau \approx 2T$ being the value that holds the resulting drop in page residency to 50%.

Only ω is needed in practice, and look-ahead is rejected because a working set changes radically whenever its process blocks for interaction. Each page-table entry carries K use bits shifted once per sampling interval $\sigma = \tau/K$; their logical OR is 1 exactly for the pages of $W(t, K\sigma)$, and a *checker* process in the running list evicts the rest. Memory management then folds into scheduling: memory demand is $\min(\omega/M, 1)$, processor demand follows from a hyperexponential model of CPU time between interactions, and a *balance* policy admits jobs so total demand matches the equipment, memory balanced first, since over-committing it produces thrashing.

Critique. The quantitative core sits in tension with the qualitative one. The traffic estimates at equations (8)–(11) assume successive interreference intervals are independent and identically distributed, whereas the prediction property, which is what makes a working set worth keeping, asserts the opposite, that "references to the same page tend to cluster in time". Denning does call these quantities estimates, but they rest on an assumption the rest of the paper denies, and they are the model's only quantitative ground.

That matters because the paper offers nothing else. Its conclusion ranks the four policies outright, "Random brings on the highest page traffic, working set the lowest", and asserts that the working set's extra cost "is more than offset by its accuracy". No experiment here tests the ranking. Denning does lean on measurement elsewhere, invoking Fine et al. and Varian and Coffman in Section 2, but the ranking itself rests on one sentence of the conclusion citing Oppenheimer and Weizer's simulation of the RCA Spectra 70/46: a citation, not a demonstration.

He also notes that with l processes on N processors, page tables are sampled every $l\sigma/N$ seconds rather than every σ, and his appendix scales the hardware timer to match; the software scheme gets no such correction, so its effective τ stretches with load, in the overload regime the model exists to handle.

2. Machine-independent virtual memory in Mach (paper 3-6)

R. Rashid, A. Tevanian, M. Young, D. Golub, R. Baron, D. Black, W. Bolosky and J. Chew, "Machine-Independent Virtual Memory Management for Paged Uniprocessor and Multiprocessor Architectures", *Proc. 2nd Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, Palo Alto, CA, October 1987, pp. 31–39.

Summary. Instruction sets had stopped obstructing portability; memory-management units had not. The authors observe that Berkeley derivatives on non-VAX hardware internally *simulate* the VAX mapping architecture, promoting one vendor's MMU into a portability specification. Mach's answer is to hold every fact about virtual memory in machine-independent structures and confine machine dependence to one module, `pmap.c`. Four structures carry the design: a resident page table treating physical memory as a cache over memory objects; an *address map* of entries mapping virtual ranges onto byte ranges of a memory object, with their protection and inheritance; the *memory object*, a file-like repository of backing store; and the *pmap*, the hardware map.

The decisive move is that the pmap holds only a *cache* of translations. Mappings may be discarded at any moment and rebuilt lazily at fault time from the machine-independent structures; only kernel mappings must be complete. That is what keeps the machine-dependent module to roughly 6K bytes on a VAX, "about the size of a device driver", while the portable

layer still supplies copy-on-write through chained shadow objects, mapped files, and external pagers running as ordinary user tasks.

The evidence is a port record: one source set runs on the VAX family, the IBM RT PC, the Sun 3, the Encore MultiMax and the Sequent Balance, with the RT PC `pmap` written in about three weeks by a programmer new to both operating systems and C. Section 5 then turns the portable base around on the hardware and finds every MMU wanting: the RT PC permits one mapping per physical page, and no multiprocessor keeps TLBs consistent.

Critique. The measurements do not test the claim they are enlisted for: what is asserted is that the machine-independent/dependent split costs nothing, what is measured is Mach against 4.3bsd, SunOS and ACIS. The wins are real but orthogonal to portability: a 256K `fork` takes 41 ms against 145 ms because of copy-on-write, not because of `pmap`. The experiment that would settle it, Mach against a hand-written machine-specific Mach, is absent. One confound they do handle: beside the 4:1 compile margin under 4.3bsd's generic buffers they report a 400-buffer run at 23 against 28 seconds.

The same gap reopens in Section 5. The RT PC's single-mapping restriction forces shared pages to be remapped on each alternation between tasks, and the evidence that these faults stay "rare enough in normal application programs" is again an end-to-end win, over IBM's ACIS 4.2a, which no more separates that cost than the fork figures do. The fault rate itself is never reported.

Section 3.5 names shadow chains as the design's main source of complexity. Mach does collapse them, garbage-collecting a shadow once the intermediate is recognised as redundant, and the authors are candid that the collector is incomplete: chains "sometimes occur during periods of heavy paging and cannot always be detected on the basis of in memory data structures alone". The discussion turns from there to that collector's locking rules, leaving the admitted residue with no bound on chain length and no cost, while every benchmark runs with memory to spare.

3. Transparent superpage support (paper 3-4)

Juan Navarro, Sitaram Iyer, Peter Druschel and Alan Cox, "Practical, Transparent Operating System Support for Superpages", *Proc. 5th USENIX Symposium on Operating Systems Design and Implementation (OSDI '02)*, Boston, MA, December 2002, pp. 89–104.

Summary. TLB coverage as a fraction of main memory fell by about a factor of 100 per decade between 1986 and 2001, and TLB misses that cost 4–5% of runtime in the 1980s now cost 30–60%. Superpages restore coverage, but a single large page size inflates footprints and paging granularity, so the operating system must mix sizes, and mixing sizes fragments the very contiguity large sizes need. The paper's answer is to treat contiguity as a contended resource.

On a fault the system reserves a contiguous aligned extent from a buddy allocator, as large as fits inside the faulting memory object without overlapping an existing reservation. Promotion is incremental: an aligned subregion is promoted once fully populated, then re-promoted at each larger size, and demoted again when the page daemon evicts one of its base pages. Under pressure, reservations are preempted and the page daemon wakes on low contiguity as well as low memory, preferentially reclaiming inactive pages that restore contiguity. Because the hardware provides one reference bit and one dirty bit per superpage, the system also demotes speculatively to find unused base pages, and demotes a clean superpage on first write, re-promoting only when every base page is dirty.

In 3500 lines on FreeBSD/Alpha, 18 of 35 benchmarks gain over 5% and 10 over 25%, and multiple sizes beat any single size (*mcf* 1.68 against 1.31 at best); after a web workload

fragments memory, the contiguity-aware daemon lets FFTW reach a 55% speedup in two runs, where a cache-only scheme obtains no superpage at all.

Critique. The results are tied to one machine, in both directions. The Alpha's four page sizes are what the incremental design exploits, and Table 2 shows what survives with only the largest: *C4* falls from 1.36 to 0.98, *SP* from 1.19 to 0.98, *mcf* from 1.68 to 1.22. Section 2.2 names the i386 family as offering exactly two sizes and never returns to what that implies for the era's dominant platform. Conversely the overheads are inflated by Alpha PTE replication, charged with 7.2 of the adversary's 8.9 percentage points.

The abstract's "these benefits are sustained even under stressful workload scenarios" is likewise broader than the evidence: the long-term case replays one web access log and runs one benchmark four times, and the concurrent experiment substitutes a dummy allocator and disables promotion altogether, measuring contiguity restored rather than speedup sustained.

Demoting on write is a frank retreat and the authors say so, that it "may deny the benefits of superpages to processes that do not write to all of the base pages". The weaker move is the alternative they discard: hashing base pages to find which were really modified was rejected on preliminary microbenchmarks costing up to 15%, and 60% on a saturated sequential read, yet the same paragraph lists two optimisations, hashing only partially dirty superpages and moving the work into the idle loop, that attack exactly that cost and are left to future work. An option dismissed on its least optimised form has not been evaluated.

Ex. 3.10 (20 pts) — three papers found by search

The three papers below come from Google, Meta and Microsoft, and each answers a question the 1968–2002 literature never faced: DRAM is now the largest single line in server cost, so which pages leave it, and on what evidence? The first two report fleet-wide deployments; the third is a design study on emulated hardware. They also disagree with one another.

1. Software-defined far memory at Google

Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, Greg Thelen, Kamil Adam Yurtsever, Yu Zhao and Parthasarathy Ranganathan, "Software-Defined Far Memory in Warehouse-Scale Computers", *Proc. 24th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, Providence, RI, April 2019, pp. 317–330. Chosen because it is the first end-to-end report of a cold-page policy running on a whole fleet rather than a testbed.

Summary. A cheap, slow memory tier below DRAM only pays if cold pages can be found without slowing anything down, and a fleet measurement shows why fixed hardware will not do it: at a cold-age threshold of $T = 120\,\mathrm{s}$, 32% of memory is cold on average but the per-machine figure ranges from 1% to 52% within a single cluster, so any provisioned far-memory capacity would be stranded on some machines and exhausted on others. The authors therefore build the tier in software, from `zswap`, which keeps compressed pages in DRAM. Upstream `zswap` fires only on direct reclaim, making its cost unbounded and bursty, so the contribution is a control plane that compresses proactively instead. A kernel daemon `kstaled` sweeps page-table accessed bits every 120 s and keeps an 8-bit age per page inside `struct page`; a second daemon `kreclaimd` compresses pages older than a per-job threshold. The threshold is chosen against a service-level objective expressed in an application-agnostic indicator: the *promotion rate*, or swap-ins per minute, held below 0.2% of the job's working set per minute. A node agent recomputes the smallest threshold meeting that objective each minute and applies the Kth

percentile of recent history. The two remaining parameters are tuned offline by replaying the per-job histograms through a model of the whole fleet and searching with a Gaussian Process Bandit. In production since 2016, the system compresses pages 3× at the median with 6.4 μs decompression, raises cold-memory coverage from 15% to 20%, and saves 4–5% of DRAM total cost of ownership.

Critique. The paper's own motivation sets the bar its evidence cannot quite clear: slowdowns "of even a few percentage points" would "offset all potential TCO savings", yet the strongest quantitative performance result is that Bigtable's IPC difference under A/B test is "within noise" — noise the authors immediately say is expected to be large in a warehouse-scale A/B. Fleet-wide application metrics are reported as showing no statistically meaningful difference, but no resolution is given for that null result, and the resolution is precisely what the 4–5% argument depends on. The accounting is similarly uneven. The quoted CPU overhead, 0.01% and 0.09% of job cycles, covers compression and decompression only; `kstaled`'s continuous scan is reported separately as "less than 11% of a single logical CPU core", in units that never enter the TCO arithmetic. Finally, K and S are never given numeric values anywhere in the paper. The 30% autotuner gain is checkable in itself, being 15% coverage hand-tuned against 20% tuned, but without the two configurations there is no way to tell whether the hand-tuned baseline was a fair opponent, or to reproduce either setting.

2. Transparent memory offloading at Meta

Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang and Dimitrios Skarlatos, "TMO: Transparent Memory Offloading in Datacenters", *Proc. 27th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, Lausanne, Switzerland, February–March 2022, pp. 609–621. Chosen because it attacks the previous paper directly, with a measurement rather than an argument, and because its mechanisms are upstream in Linux.

Summary. TMO's target is the same cost, but it rejects Google's control signal. The claim is empirical: on the Web application, the host with the *faster* SSD backend shows a *higher* promotion rate and simultaneously serves more requests per second, so swap-in rate is not monotone in application harm, because it ignores both the speed of the backend and the benefit of the memory freed. In its place TMO introduces Pressure Stall Information, a kernel mechanism that measures lost work directly by timing how long processes are stalled on a resource, reported per container as `some` (at least one process stalled) and `full` (all of them). A userspace agent, Senpai, applies the control law $reclaim_mem = current_mem \times r \times \max(0,\, 1 - \mathrm{PSI}_{some}/\mathrm{PSI}_{thresh})$ every six seconds with $r = 0.0005$ and $\mathrm{PSI}_{thresh} = 0.1\%$, so reclaim slows as pressure approaches the target and stops above it. Two kernel changes make the pages available: shadow entries record a page's reuse distance so genuine refaults can be distinguished from first touches, and reclaim is rebalanced to swap anonymous memory as soon as the file cache working set starts refaulting, instead of treating swap as emergency overflow. TMO also offloads the "memory tax" of logging, routing and other sidecars, whose service-level agreements are looser. Across millions of servers it reports 20–32% of total memory saved, with either SSD or `zswap` as the backend.

Critique. That headline is not a measured quantity. It adds the 7–19% of Figure 9 to the 13% of Figure 10, and the paper gives two incompatible accounts of what the first is a fraction of: the caption says "normalized to their resident memory size", per application, while Section 4.1 says "7-19% of the total memory", which is Figure 10's per-server denominator. One of the two

is wrong, and until that is settled the sum denotes nothing. The second problem is that TMO's advertised advantage over `g-swap`, that Senpai "does not require offline application profiling", is weaker than stated. The two constants above were "iteratively arrived at" by studying production workloads, and they are global. Section 3.3 asserts that "TMO's effectiveness is not very sensitive to these parameters", and then Section 4.4 shows the opposite: Config B saves more memory than Config A at the cost of "significant RPS regression, violating our design goal". The authors do flag automated tuning as future work, so the defect is the insensitivity claim rather than a missing plan. Last, PSI's IO component treats any wait on block IO as a stall, because the hardware cannot separate oversubscription from a device's expected latency; this reads as pressure exactly where SSDs are cheapest, and the paper's only answer is that it "has worked well for us in production".

3. CXL memory pooling at Microsoft

Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura and Ricardo Bianchini, "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms", *Proc. 28th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23), Volume 2*, Vancouver, BC, March 2023, pp. 574–587. Chosen because it moves the problem out of the kernel's reclaim path entirely and into hardware the other two papers could only wish for.

Summary. In a public cloud the waste is not cold pages but *stranded* ones: traces from 100 Azure clusters show that when all cores of a server are rented, leftover memory cannot be rented, reaching 25% at the 95th percentile. Pooling would fix this, but cloud VMs pin their whole address space so that virtualisation accelerators work, which rules out the page-fault-based schemes of the two papers above. Pond uses Compute Express Link instead, giving cacheable load/store access to an external memory controller shared by 8–16 sockets, a size chosen because wider pools add switches and retimers for diminishing benefit: 8–16 sockets add 70–90 ns over NUMA-local DRAM, rack-scale pooling more than 180 ns. Latency still matters: of 158 workloads under a 182% latency increase, 43% stay within 5% of local-memory performance while 21% lose more than 25%. Two ideas rescue it. Half of all VMs never touch half their rented memory, and that memory can be exposed as a *zNUMA* node, memory but no cores, which the guest's own NUMA policy avoids; measured traffic to it is 0.06–0.38%. Second, two models, a random forest over hardware counters for latency insensitivity and a gradient-boosted quantile regression over VM metadata for untouched memory, predict how much pool memory each VM can take, with a monitor that migrates mispredicted VMs back to local DRAM. The result is 7% less DRAM at a 16-socket pool, or 3.5% of server cost.

Critique. No CXL hardware is involved: the external memory controller is a paper design, its latency is emulated by disabling a second socket's cores, and the savings come from a capacity-only simulator. That is stated plainly, but one consequence is structural: a real 16-socket pool has sixteen hosts contending for one controller's DDR5 channels, whereas the emulation puts a single host on the remote socket, so interference inside the pool is never measured; the paper's answer is that more controllers can be attached. Section 7 does raise gaming, replying that "any individual customer's behavior will have a small impact" across millions of customers. But the untouched-memory model's most important feature is percentiles of untouched memory from *that customer's own* VMs last week, so a customer who touches everything moves their own prediction and nobody else's; the aggregate reply misses the mechanism, and the section's other answer, discounting under-used resources, is a clause the paper never implements or evaluates. The quoted 1–5% degradation likewise holds only inside the configured tail: 2% of VMs exceed

the margin and the monitor mitigates at most half of them. What the rest lose is unbounded, with Finding 3 reporting some workloads down 30–35% once a fifth of their memory spills. Pond bounds how many VMs miss the margin, not by how much.